

TraceLens 기획안 용어 해설

기획안 v3를 읽을 때 옆에 두는 문서. 파이프라인 순서대로 정리했고, 각 용어는 "한 줄 정의 → 왜 필요한가 → 예시" 순으로 설명한다.

1. 실행·추적 관련 (Tracer)

트레이스 (Trace) / TraceEvent

코드를 실행하면서 "무슨 일이 일어났는지"를 순서대로 기록한 일지. 우리 제품의 원재료다. TraceEvent는 그 일지의 한 줄 — "3번 줄이 실행됐다", "함수 f가 호출됐다", "변수 x가 5가 됐다" 같은 기록 하나하나를 말한다.

실행 훅 (sys.settrace / sys.monitoring)

Python이 "한 줄 실행할 때마다 나한테 알려줘"라고 등록할 수 있는 공식 장치. 둘 다 Python 내장 기능이 고 하는 일은 같은데, settrace는 오래됐고 단순하며, monitoring은 Python 3.12에 새로 생겼고 빠르다. 단, 둘 다 "3번 줄 실행했음"이라는 사건만 알려주지 변수 값까지 챙겨주진 않는다 — 그래서 아래 스냅샷이 필요하다.

스냅샷(snapshot)과 diff

스냅샷 = 그 순간 변수·객체 상태를 사진 찍듯 저장한 것. diff = 직전 사진과 비교해서 달라진 부분만 추린 것. 매 줄마다 모든 변수를 다 기록하면 데이터가 폭발하니까, "이번 줄에서 x만 3→4로 변했다"처럼 변화분만 기록한다.

직렬화 (serialization) / bounded serializer

직렬화 = 메모리 속 Python 객체를 JSON 같은 저장 가능한 형태로 바꾸는 것. bounded(제한된) serializer는 안전장치가 달린 직렬화기다: 리스트가 백만 개짜리면 앞부분만, 객체가 객체를 무한히 물고 있으면(순환 참조) 적당히 끊고, 깊이도 제한한다. 안 그러면 사진 한 장이 수십 MB가 된다.

observationally passive (부작용 없는 관찰)

"관찰하는 행위가 프로그램을 바꾸면 안 된다"는 원칙. 함정: Python에서 객체를 문자열로 바꾸는 `repr()`이나 속성을 읽는 `getattr()`은 사용자가 정의한 코드를 실행할 수 있다. 우리가 상태를 구경하려다 사용자 코드의 동작이 달라지면 "실제 실행을 보여준다"는 약속이 깨진다. 그래서 안전한 타입(숫자, 리스트, dict...)만 직접 읽고, 수상한 객체는 `UnsupportedObject`로 뭉뚱그린다.

frame / frameId / 호출 스택

frame = 함수가 한 번 호출될 때 생기는 작업 공간(지역변수 보관함). 호출 스택 = 그 작업 공간이 쌓인 탑. `f()`가 `g()`를 부르면 frame이 2층으로 쌓인다. frameId는 각 호출에 붙이는 고유 번호 — 재귀에서 `f`가 5

번호출되면 함수 이름은 다 같아서 구분이 안 되니, f1.f2.f3처럼 번호로 구분한다.

seq

모든 **TraceEvent**에 붙는 **일련번호 (1, 2, 3, ...)**. "실행 순서"의 기준. 대본이 "seq 42 시점을 보여줘"라고 참조할 때 쓴다.

source-line boundary와 귀속 (causedByLine)

"변수가 변한 걸 언제 알아채고, 어느 줄 탕으로 기록할 것인가"의 규칙. 함정: 실행 혹은 그 줄을 실행하기 직전에 알려준다. `x = x + 1` 줄에 도착한 순간엔 아직 `x=1`이고, `x=2`가 된 건 다음 줄에 도착해서야 발견된다. 그래서 "다음 줄에서 발견한 변화는 직전 줄(causedByLine) 때문"이라고 귀속시키는 규칙을 미리 못 박는 것이다. 안 그러면 화면 하이라이트가 한 줄씩 밀린다.

mutation / 합성 이벤트 (synthetic event)

mutation = 객체 내용이 바뀌는 것 (`a.append(3)` 등). **합성 이벤트** = **혹이 알려주지 않아서 우리가 직접 만들어내는 이벤트**. Python은 "리스트가 변경됨"이라는 알람을 주지 않는다. 스냅샷 비교로 "어? [1,2]였는데 [1,2,3]이네" 하고 우리가 추론해서 mutation 이벤트를 만들어야 한다. 이게 기획안이 "최대 기술 난제"라고 부르는 부분이다.

aliasing (별칭) / objectId

aliasing = 두 변수가 같은 객체를 가리키는 것. `b = a`는 복사가 아니라 같은 리스트에 이름표를 하나 더 붙인 것이다. 이걸 화면에 그리려면 값("[1,2]")만으론 부족하고 "a와 b가 가리키는 게 같은 물건인지"를 알아야 한다. 그래서 모든 객체에 고유번호(objectId)를 붙이고, 변수는 그 번호를 참조한다. AI 코드에서 학생이 가장 많이 데이는 부분이라 우리 시각화의 핵심 소재다.

2. 압축·요약 관련 (Digest)

Digest / 루프 접기

Digest = 수만 개 **TraceEvent**를 LLM이 읽을 수 있는 분량으로 줄인 요약본. 루프 접기는 그 핵심 기술: 1000번 도는 반복문을 "이 구간이 1000번 반복됨, 매번 i가 1씩 증가"로 접는 것. 단, 특정 회차에서만 특별한 일이 생기면(if에 처음 걸리는 순간 등) 그 회차는 접지 않고 살린다 — 그게 "interesting iteration".

DigestSpan / spanRef

DigestSpan = 요약본의 각 덩어리가 원본 일지의 몇 번~몇 번(seq)에 해당하는지 적어둔 딱지. **spanRef** = 그 딱지의 이름표 (예: "loop_3"). 왜 필요하냐면: LLM에게 "seq 142부터 286까지 보여줘"라고 숫자를 쓰게 하면 숫자를 지어낼 위험이 있다. 대신 "loop_3 구간을 보여줘"라고 이미 존재하는 딱지 중에서 고르게만 하면 지어낼 수가 없다. 객관식만 허용하고 주관식을 금지하는 것.

Resolver

LLM이 고른 `spanRef("loop_3")`를 실제 `seq` 구간([142, 286])으로 바꿔주는 번역기. "결정적 (deterministic)"이라는 말은 같은 입력이면 항상 같은 출력이 나온다는 뜻 — LLM처럼 그때그때 다르지 않다.

3. 연출·대본 관련 (Director)

Screenplay (연출 대본)

LLM이 작성하는 "영상 편집 지시서". 챕터(대단원) → 장면(scene) 구조로, 각 장면마다 "어느 구간(`spanRef`)을, 어떤 그림(프리미티브)으로, 누구를 주인공(`focus`)으로, 어떤 속도(`pacing`)로, 무슨 자막(`narration`)과 함께" 보여줄지 적혀 있다. 값 자체는 못 적는다는 게 핵심 규칙.

템플릿 + 바인딩 (template + bindings)

자막에 값을 직접 쓰지 못하게 하는 장치. LLM은 "{x}가 갱신됩니다"라는 빈칸 있는 문장(템플릿)과 "그 빈칸은 `seq 82`의 `count` 변수 값으로 채워라"(바인딩)만 쓸 수 있다. 실제 숫자는 재생기가 트레이스에서 꺼내 채운다. LLM이 "x가 7이 됩니다"라고 거짓말할 경로 자체를 없애는 것.

JSON Schema 검증

LLM의 출력이 약속된 형식을 지켰는지 자동 검사하는 것. JSON Schema는 "chapters는 배열이어야 하고, 각 scene엔 `spanRef`가 반드시 있어야 하고..." 같은 규칙 명세서다. LLM 출력이 이걸 통과 못 하면 재시도시키고, 그래도 안 되면 폴백으로 넘어간다.

폴백 (fallback)

주력 수단이 실패했을 때 자동으로 넘어가는 대비책. 우리 설계엔 두 겹 있다: ① LLM이 실패하면 → 규칙 기반 대본 생성기(LLM 없이 코드 하이라이트+변수 변화를 순서대로 트는 기본 무비) ② 특정 값을 그릴 전용 그림이 없으면 → Generic State View(이름/타입/값 표로라도 보여줌). "실패해도 빈 화면은 없다"를 만드는 구조.

관찰형 vs 해석형 내레이션

관찰형 = "count가 3에서 4로 변경됩니다" (트레이스로 증명 가능한 사실). 해석형 = "이 변수가 최솟값 후보를 추적합니다" (LLM의 해석 — 그럴듯하지만 틀릴 수 있음). 장면 자막은 관찰형 중심으로 하고, 해석형은 챕터 제목·요약에만 허용하는 게 우리 절충안이다.

4. 재생 관련 (Player)

프리미티브 (primitive)

시각화의 기본 부품. 리스트를 그리는 칸 상자, 호출 스택 카드, 객체 참조 그래프 등 "그림의 종류". 레고 블록 5종을 만들어두고, 대본이 "이 장면은 3번 블록으로"라고 고르는 방식이다.

GSAP / Timeline / 스크립(scrub)

GSAP = 웹 애니메이션 라이브러리 (현 프로토타입에서 이미 사용 중). **Timeline** = 여러 애니메이션을 한 줄로 이어붙인 편집 트랙. 트랙 하나로 묶으면 재생/일시정지/배속이 공짜로 생기고, 스크립 = 유튜브처럼 진행바를 드래그해서 아무 시점으로 이동하는 것도 된다.

semantic timeline

"애니메이션 시간"과 "프로그램 시간(seq)"을 분리해서 연결한 표. 문제 상황: 막대 두 개가 자리를 바꾸는 애니메이션 0.5초 중간에 일시정지하면, 인스펙터는 교환 전 값을 보여줘야 하나 후 값을 보여줘야 하나? 답을 정하는 장치가 이것 — 화면이 몇 초든 "마지막으로 확정된 seq는 82"를 따로 기억해서, 인스펙터는 항상 seq 82의 일관된 상태를 보여준다.

인스펙터 (inspector)

일시정지했을 때 그 순간의 변수·객체를 들여다보는 패널. "영상은 못 하는 것"이 우리 차별점인 이유.

5. 실행 환경·보안 관련

Pyodide / WASM

Pyodide = 브라우저 안에서 도는 **Python**. 원래 Python은 서버나 내 컴퓨터에 설치해야 하는데, WASM(웹 어셈블리 — 브라우저가 실행할 수 있는 기계어 비슷한 형식)으로 통째로 변환한 것이 Pyodide다. 덕분에 서버 없이 사용자 브라우저에서 코드를 실행한다 → 서버 해킹 걱정과 서버 비용이 크게 준다.

Web Worker

브라우저의 백그라운드 작업 스레드. 무거운 일(Python 실행)을 화면 담당 스레드와 분리해서, 사용자 코드가 3초를 돌아도 UI가 안 멈추게 한다.

chunk flush

수집한 트레이스를 다 끝나고 한 번에 넘기지 않고, 중간중간 나눠서(chunk) 미리 보내두는(flush) 것. 이유: 무한 루프에 걸려 Worker를 강제 종료하면 그 안에 있던 기록이 통째로 사라진다. 중간중간 내보내두면 "죽기 직전까지의 기록"은 살아서 시각화할 수 있다.

콜드 스타트 / 워밍 스타트

콜드 = 처음이라 준비물(Pyodide 런타임 수십 MB)부터 다운로드해야 하는 상태. 워밍 = 이미 준비돼서 바로 실행되는 상태. 콜드가 수 초~십수 초 걸릴 수 있어서, 페이지 열리자마자 백그라운드에서 미리 받아두는 걸로(선로딩) 흡수한다.

SharedArrayBuffer / COOP-COEP

실행 중인 Python을 중간에 멈추려면(interrupt) 스레드 간 공유 메모리(SharedArrayBuffer)가 필요한데, 브라우저가 보안상 이것을 잠가놔고, 열려면 서버가 특수 헤더(COOP/COEP) 두 개를 보내야 한다. 문제는 이 헤더가 배포 방식에 제약을 건다는 것 — 그래서 "배포 환경 포함 스파이크로 검증"이라고 적혀 있다. 지금 다 이해할 필요는 없고 "중단 버튼 구현엔 배포 설정 이슈가 달려온다" 정도면 충분하다.

untrusted code (신뢰할 수 없는 코드) / 위협 모델

우리 사용자는 정의상 "이해 못 한 코드"를 들고 온다 = 우리는 의도적으로 수상한 코드를 실행해주는 서비스다. 위협 모델은 "공격자가 뭘 할 수 있고 우리는 뭘 막아야 하는가"의 목록. 핵심 원칙: 실행 공간(Worker)에는 훔쳐갈 비밀(API 키 등)을 아예 두지 않는다.

CSP / connect-src

웹페이지가 접속할 수 있는 외부 주소를 서버가 화이트리스트로 제한하는 브라우저 보안 기능. 사용자 코드가 몰래 어딘가로 데이터를 보내는 걸 브라우저 차원에서 차단한다.

서버리스 프록시 (serverless proxy)

서버리스 = 서버를 상시 운영하지 않고, 요청이 올 때만 잠깐 실행되는 함수 (Vercel Functions 등). 프록시 = 중계자. LLM API 키를 브라우저에 두면 누구나 훔쳐 쓸 수 있으니, 키는 이 중계 함수에만 두고 브라우저는 중계 함수를 거쳐 LLM을 호출한다. 우리 서버 구성 요소는 이거 하나다.

opt-out / local-only

opt-out = 사용자가 "이 기능 끄래"를 선택할 수 있게 하는 것. 코드가 외부 LLM으로 전송되는 게 싫은 사용자는 LLM을 끄고 규칙 기반 무비만 쓸 수 있게 한다(local-only).

6. 개발 방법론 관련

MVP (Minimum Viable Product)

"이 제품이 성립한다"를 증명하는 최소 범위. 완성형이 아니라 증명용. 우리 MVP = 단일 파일 Python + 프레임워크 5종 + SVG 렌더러.

수직 슬라이스 (vertical slice) / Slice 0-1-2

모든 기능을 조금씩이 아니라, 하나의 경로를 처음부터 끝까지 관통시키는 개발 방식. 케이크를 층별로 다 만들고 쌓는 게 아니라, 한 조각을 위에서 아래까지 잘라 먼저 완성하는 것. Slice 0 = 가짜 데이터로 재생기만 검증 → Slice 1 = 진짜 실행 + 규칙 대본으로 완주(LLM 없이) → Slice 2 = LLM을 엮음. 단계마다 "어디가 고장났는지"를 분리할 수 있다.

fixture

테스트용으로 미리 준비해둔 고정 데이터 파일. 예: 진짜 실행으로 뽑아 저장해둔 트레이스 JSON, 손으로 쓴 대본 JSON. 렌더러 담당은 Tracer가 완성되길 기다릴 필요 없이 fixture를 입력 삼아 개발한다 — 3인 병렬 개발의 핵심 장치.

스파이크 (spike)

본격 구현 전에 "이게 기술적으로 되긴 하나?"를 며칠 안에 확인하는 일회용 실험 코드. 결과물은 제품 코드가 아니라 판단(Go/No-Go)이다. 예: "Pyodide에서 settrace로 변수 수집이 실제로 되는가?"

Go/No-Go

스파이크의 결론 형식 — "간다/안 간다"의 명시적 판정. 애매하게 "될 것 같다"가 아니라 기준을 정해두고 통과/탈락을 가른다.

ablation (절제 실험)

핵심 부품을 뺀 버전과 비교해서 그 부품의 기여도를 측정하는 실험. 우리 경우: LLM 뺀 무비(규칙 기반) vs LLM 넣은 무비를 비교해 "LLM이 정말 설명을 좋게 만드는가"를 숫자로 증명한다. 심사에서 "LLM 없어도 되는데 왜 AI 프로젝트냐"는 질문의 방어 무기.

벤치마크 / 층화 (stratification)

벤치마크 = 성능·성공률을 재는 표준 시험 문제집. 층화 = 문제를 유형별로 나눠서 채점하는 것. "전체 성공률 87%"보다 "재귀는 95%인데 클래스는 60%"가 훨씬 유용한 정보다.

P50 / P95

응답 시간 통계. P50 = 사용자 절반이 경험하는 시간(중간값), P95 = 95%의 사용자가 이 시간 안에 끝남(= 최악에 가까운 5%의 경계). 평균은 극단값에 왜곡되니 실무에선 이 두 개를 쓴다. "평균 3초"여도 P95가 20초면 20명 중 1명은 20초를 기다린다는 뜻.

acceptance criteria (합격 기준)

"성공했다"를 판정하는 사전 합의된 숫자 기준. 예: "대본 검증 통과율 90% 이상". 우리는 근거 없이 지금 숫자를 정하지 않고, 스파이크로 현재 수준(baseline)을 잰 뒤 정하기로 했다.

7. 실험 설계 관련

between-subject / within-subject (crossover)

between = 참가자를 그룹으로 나눠 각자 다른 조건 하나만 경험 (A그룹은 텍스트, B그룹은 우리 도구).

within(crossover) = 한 사람이 모든 조건을 순서 바꿔가며 경험. between은 깔끔하지만 사람이 많이 필요하고, within은 적은 인원으로 되지만 순서 효과를 통제해야 한다. 그래서 "확보 가능한 표본 수를 먼저 계산한 뒤 선택"이라고 적혀 있다.

transfer question (전이 문항)

본 것을 기억하는지가 아니라, 이해를 새 상황에 적용할 수 있는지 묻는 문제. "최종 x 값은?"(암기)이 아니라 "입력이 [1,2,3]에서 [3,1]로 바뀌면 결과가 어떻게 될까?"(이해). 우리 실험의 문항 원칙.

exploratory study (탐색적 연구)

표본이 작을 때 "증명했다" 대신 "이런 경향을 관찰했다"로 주장 수위를 낮추는 연구 형식. 통계적으로 정직한 표현 방식이며, 심사에서 과장 지적을 피하는 안전장치.

예비 실험 (pilot)

본 실험 전에 소수 인원으로 절차를 리허설하는 것. 목적은 효과 검증이 아니라 "문제가 너무 어렵진 않나, 30분 안에 끝나나, 로그는 잘 쌓이나"의 확인.